CSE 211 — Theory of Computation

Turing Machines — Definition and Design

Lesson notes · Exam approach · TF practice with solutions · Quick notes (Atif Sir's portion)

How to Read These Notes

EXAM FOCUS	Exam importance, where it appears in the TF collection
MEMORIZE	Rules, tables and formulas to memorize
TRAP	Common mistakes and exam traps
EXAM-READY ANSWER	Model wording you can write in the exam
NOTE	Extra explanation / clarification
Grey box, monospace	Code, worked tables, derivations and hand-drawn sketches
Centered grey box	Formula to remember / final numerical result
Purple heading / ✓ green	Practice questions / their solutions

Contents

Exam Map — What the TF Papers Ask — TM design: one 7–20 mark question per paper
Lesson 11 — The Turing Machine: Definition, IDs, Language — 7-tuple, tape, moves, ID notation, halting, RE languages
Lesson 12 — Designing Language-Deciding TMs — marking technique: 0^n1^n, a^nb^nc^n, w−w, equal counts
Lesson 13 — TMs Computing Functions — monus, unary subtraction, increment, parity output
Quick Notes — TM Basics — revision bullets + key diagrams

NOTE Slide numbers refer to the PDF pages of Lecture - 4 (Turing Machine 1).pdf (123 pages; beamer builds). Hopcroft notation: M = (Q, Σ, Γ, δ, q0, B, F), δ(q, X) = (p, Y, D), D ∈ {L, R}, blank B (printed ⊔ in the machines below to avoid confusion with the symbol B). IDs are written X1…Xi−1 q Xi…Xn. Every TM in the solutions was run by a simulator on all strings up to length 9 (7–8 for larger alphabets) and its output compared with the specification. Bit-manipulation TMs (XOR/AND/OR/NOT/…) are in the next note. Practice provenance: Actual = taken from a TF paper; Adapted = TF question with changed details; Constructed = new question in TF style.

Exam Map — What the TF Papers Ask

Session	TM design questions from this lecture
2016-17	Q6(c) TM for strings with an equal number of 0s and 1s (10)
2015-16	Q7(a) TM for {a^n b^n c^n | n ≥ 1}; working principle (13) · Q8(a) TM computing max(b − a, 0), a and b in unary 1s separated by 0 (13)
2014-15	Q8(b) State diagram of a TM deciding {w−w | w ∈ {a,b}∗} (7)
2013-14	Q2(a) TM recognizing {0^N 1^N 0^N | N ≥ 0} (7) · Q3(a) Can a TM write blank? Can Γ = Σ? Head in same place two steps? Single state? (7)
2012-13	Q3(b) High-level description and state diagram of a TM deciding {w−w−w | w ∈ {a,b}∗} (7) · Q3(c) TM on #w: if |w| even write #0^n, else #1^n (8⅓)
2011-12	Q1(a) TM computing x − y (x ≥ y) in unary where N is written as 1^(N+1) (8⅓) · Q3(a) TM on #w: if |w| even write #0^n, else #1^n (8⅓)
2010-11	Q11(b) TM deciding {a^(2^n) b | n ≥ 0}: state diagram and configurations on aaaab (8⅓)
2009-10	Q12(a) Formal TM definition; TM that scans right until it finds two consecutive a's, then halts (alphabet {a,b,␣,▷}) (5) · Q12(b) TM deciding {a^n c^n b^n d^n : n ≥ 0} (7)
2008-09	Q3(a) Formal notation of the Turing machine (2⅓)
2007-08	Q7 TM deciding {0^n 1^n | n ≥ 1} as a transition table; IDs on input 0011 (10)
2006-07	Q10 TM deciding {0^n 1^n | n ≥ 1}; transition diagram (10) · Q12 Language of a TM; recursive language; describe L(M) for δ(q0,0)=(q1,1,R), δ(q0,B)=(qf,B,R), δ(q1,1)=(q0,0,R), δ(q1,B)=(qf,B,R) (10)

EXAM FOCUS A TM design question appears in every paper (7–20 marks). Since 2017-18 the 15-mark one is always a BIT-MANIPULATION machine (next note); before that the favourites were {0^n1^n}, {a^nb^nc^n}, {w−w}, equal 0s/1s, unary subtraction (monus) and binary increment. Learn the marking technique (Lesson 12) and be able to write the transition table and the ID trace of {0^n1^n}.

Lesson 11 — The Turing Machine: Definition, IDs, Language

Slides 1–41, 116–123

1. Why Turing Machines? (sl. 2–5)

Neither FAs nor PDAs are general models of computers — they cannot even recognise {a^nb^nc^n}. A TM (Turing, 1912–54) is a finite automaton with a single infinite tape on which it can read and write.

2. The Machine (sl. 6–26)

1. A finite control in one of finitely many states.
2. A tape of cells, each holding one symbol of a finite alphabet.
3. Initially the input (finite string over Σ) is on the tape, all other cells (infinite in both directions) hold the blank B. B is a tape symbol, not an input symbol.
4. A head scanning one cell, initially the leftmost input cell.
5. One move depends on state and scanned symbol: change state (maybe the same), write a symbol (maybe the same), move the head one cell left or right — it must move, it cannot stay.

M = (Q, Σ, Γ, δ, q0, B, F)
δ(q, X) = (p, Y, D) — next state, symbol written, direction

Γ is the set of all tape symbols with Σ ⊆ Γ and B ∈ Γ − Σ; q0 start; F accepting states.

> **EXAM-READY ANSWER** Formal notation of a TM (2008-09 Q3(a), 2009-10 Q12(a)): the 7-tuple above, with Q finite states, Σ finite input alphabet (not containing B), Γ finite tape alphabet ⊇ Σ ∪ {B}, δ : Q × Γ → Q × Γ × {L, R} a partial function (undefined = the machine halts), q0 ∈ Q start state, B ∈ Γ the blank, F ⊆ Q accepting states.

3. Instantaneous Descriptions (sl. 27–41)

After finitely many moves only finitely many cells have been visited, so an ID shows only the cells between the leftmost and rightmost non-blank (plus the scanned cell): **X1X2…Xi−1 q XiXi+1…Xn** — state q, head on Xi. ⊢ one move.

```
δ(q, Xi) = (p, Y, L):  X1…Xi−1 q Xi Xi+1…Xn  ⊢  X1…Xi−2 p Xi−1 Y Xi+1…Xn
δ(q, Xi) = (p, Y, R):  X1…Xi−1 q Xi Xi+1…Xn  ⊢  X1…Xi−1 Y p Xi+1…Xn
Boundary cases:  i = 1, move L:  q X1…Xn ⊢ p B Y X2…Xn      i = n, move R:  X1…Xn−1 q Xn ⊢ X1…Xn−1 Y
p B
A blank written at the end (or start) joins the infinite blanks and disappears from the ID.
```

4. Language, Halting, RE (sl. 116–123)

L(M) = {w ∈ Σ∗ | q0 w ⊢∗ α p β for some p ∈ F}. A TM **halts** when it is in a state scanning a symbol with no defined move. Languages accepted by TMs are the **recursively enumerable (RE)** languages; a TM that accepts L and halts on every input decides L (recursive language, Lesson 19). The monus machine (Lesson 13) was designed to compute a function, so its seventh component F is omitted; it halts on every input (sl. 121–123).

EXAM-READY ANSWER Answers to the 'TM facts' questions (2013-14 Q3(a)): (i) Can a TM write the blank? Yes — Γ contains B and δ may write it (e.g. erasing a symbol); it simply must not be an input symbol. (ii) Can Γ = Σ? No — B ∈ Γ but B ∉ Σ, so Γ ⊋ Σ always. (iii) Can the head be at the same location in two successive steps? No in the basic model: every move is L or R (the S 'stay' option exists only in multitape TMs and does not add power). (iv) Can a TM have a single state? Yes: a one-state TM is legal (the state is both start and accepting, or just a halting state). It can only apply the same rule to every scanned symbol, so it recognises trivial languages: if the single state q0 is accepting, the start ID q0w already has an accepting state, so L = Σ∗; if it is not accepting, no ID ever has one, so L = ∅. So yes, it is allowed, but it is useless for anything else — a single state cannot count or compare.

Exam Approach

1. Formal parts: list Q, Σ, Γ, δ (as a table), q0, B, F.
2. Tracing: write each ID as left-tape · state · scanned symbol · right-tape, one line per move.
3. 'Describe L(M)' for a given rule set: follow the loop of states on a few strings and generalise.

Practice — Lesson 11

(a) Actual — 2006-07 Q12. (b) Actual — 2007-08 Q11. (c) Actual — 2013-14 Q3(a). (d) Actual — 2008-09 Q3(a).

(a) What is the language of a Turing machine? Define recursive language. M = ({q0,q1,qf}, {0,1}, {0,1,B}, δ, q0, B, {qf}) with δ(q0,0) = (q1,1,R), δ(q0,B) = (qf,B,R), δ(q1,1) = (q0,0,R), δ(q1,B) = (qf,B,R). Describe L(M). (10)

(b) Recursive, RE and non-RE languages? M as above with δ(q0,0) = (q0,B,R), δ(q0,1) = (q1,B,R), δ(q1,1) = (q1,B,R), δ(q1,B) = (qf,B,R). Describe L(M). (6 + 4)

(c) Answer and explain: can a TM ever write the blank symbol? Can Γ equal Σ? Can the head be in the same cell in two successive steps? Can a TM have just one state? (7)

(d) Give the formal notation of the Turing machine. (2⅓)

✓ Solution

(a) L(M) = {w | q0w ⊢∗ α qf β}. Recursive: L = L(M) for a TM that halts on every input. For the given M: q0 reads 0 → q1 (writes 1), q1 reads 1 → q0 (writes 0): the machine alternates, consuming 0, 1, 0, 1, …; it accepts in qf when it meets the blank in q0 (after an even number of symbols) or in q1 (after an odd number). **L(M) = (01)∗(0 ∪ ε)** — strings alternating 0,1,0,1,… starting with 0 (ε, 0, 01, 010, 0101, …); any other string reaches a state with no move and halts non-accepting. (The tape ends up complemented, but the language is what is accepted.) Check: 011: q0 0→q1, q1 1→q0, q0 1: no move → reject.

(b) Definitions: Lesson 19. M erases a block of 0's in q0 (writing B), then on the first 1 goes to q1 and erases 1's, accepting at the blank. **L(M) = $0*1^+$ = {0^m1^k | m ≥ 0, k ≥ 1}** — ε is rejected (q0 on B has no move). Checked by simulation on all strings up to length 9.

(c) EXAM-READY ANSWER in §4; shortest form: yes / no / no / yes (single state both start and accepting; e.g. accepts {w} only trivially).

(d) EXAM-READY ANSWER in §2.

Lesson 12 — Designing Language-Deciding TMs

Slides 42–70 + TF problems

1. The Marking Technique: {0^n1^n | n ≥ 1} (sl. 42–65)

Repeatedly change the leftmost 0 to X, move right over 0s and Ys to the first 1, change it to Y, move back left to the rightmost X, step right and repeat. When no 0 is left (the next symbol is Y), check that only Ys remain up to the blank. The tape visited always matches X∗0∗Y∗1∗ (sl. 47). Any other shape ⇒ no move ⇒ halt without accepting.

$$M = (\{q_0, q_1, q_2, q_3, q_4\}, \{0, 1\}, \{0, 1, X, Y, B\}, \delta\ , q_0, B, \{q_4\})$$

where δ is given by the table.

	Symbol				
State	0	1	X	Y	B
q_0	(q_1, X, R)	–	–	(q_3, Y, R)	–
q_1	$(q_1, 0, R)$	(q_2, Y, L)	–	(q_1, Y, R)	–
q_2	$(q_2, 0, L)$	–	(q_0, X, R)	(q_2, Y, L)	–
q_3	–	–	–	(q_3, Y, R)	(q_4, B, R)
q_4	–	–	–	–	–

Figure 8.9: A Turing machine to accept $\{0^n1^n \mid n \geq 1\}$

Figure 12.1 — Transition table of the 0^n1^n TM (sl. 46)

```
TM for {0^n1^n | n ≥ 1}:
δ(q0, 0) = (q1, X, R)
δ(q0, Y) = (q3, Y, R)
δ(q1, 0,Y) = (q1, same, R)
δ(q1, 1) = (q2, Y, L)
δ(q2, 0,Y) = (q2, same, L)
δ(q2, X) = (q0, X, R)
δ(q3, Y) = (q3, Y, R)
δ(q3, ⊔) = (q4, ⊔, R)
start q0; accepting/halting ['q4']; ⊔ = blank; 'same' = write the symbol read; q0 = start of a round, q1 =
seek a 1, q2 = return, q3 = final check
```

```
IDs on 0011:
[q0]0011 ⊢ X[q1]011 ⊢ X0[q1]11 ⊢ X[q2]0Y1 ⊢ [q2]X0Y1 ⊢ X[q0]0Y1 ⊢ XX[q1]Y1 ⊢ XXY[q1]1 ⊢ XX[q2]YY ⊢
X[q2]XYY
⊢ XX[q0]YY ⊢ XXY[q3]Y ⊢ XXYY[q3]⊔ ⊢ XXYY⊔[q4]⊔
```

On 0010 the machine reaches X X Y 0 q1 ⊔ and has no move for ⊔ in q1 (sl. 64–65) — rejected.

We have represented δ as a transition diagram.

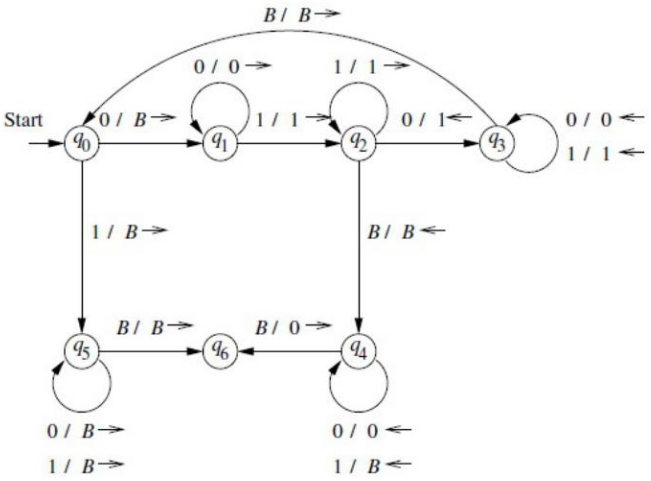


Figure 12.2 — Transition diagram of the same TM; arcs X/YD (sl. 105)

Diagram notation (sl. 66–67): an arc q → p labelled X/YD means δ(q, X) = (p, Y, D); ← is L, → is R; double circle = accepting; the blank symbol is B unless stated otherwise.

Exam Approach — Deciding a Language

1. Describe the working principle in 2–4 sentences first (the examiner gives marks for it).
2. Choose marking symbols; one round = one matched group; state what the tape looks like between rounds.
3. Give the table / diagram; make every non-accepting situation simply have no move.
4. Trace one member (IDs) and one non-member.

Practice — Lesson 12

(a) Actual — 2006-07 Q10, 2007-08 Q7, 2011-12-style. (b) Actual — 2015-16 Q7(a). (c) Actual — 2013-14 Q2(a). (d) Actual — 2009-10 Q12(b). (e) Actual — 2016-17 Q6(c). (f) Actual — 2014-15 Q8(b), 2012-13 Q3(b). (g) Actual — 2010-11 Q11(b). (h) Actual — 2009-10 Q12(a).

(a) Build a TM for {0^n1^n | n ≥ 1} (or ≥ 0); transition diagram/table; IDs on 0011. (10)

(b) Design a TM for {a^nb^nc^n | n ≥ 1}; briefly describe the working principle. (13)

(c) Build a TM to recognise L = {0^N1^N0^N | N ≥ 0}. (7)

(d) TM deciding {a^nc^nb^nd^n | n ≥ 0}. (7)

(e) Define a TM for the set of strings with an equal number of 0s and 1s. (10)

(f) Draw the state diagram of a TM that decides {w−w | w ∈ {a,b}∗}; and a high-level description with state diagram for {w−w−w | w ∈ {a,b}∗}. (7)

(g) State diagram and configurations (on aaaab) of a TM deciding {a^(2^n) b | n ≥ 0} (the input aaaab has four a's). (8⅓)

(h) Write out in full a TM that scans right until it finds two consecutive a's and then halts, alphabet {a, b, ⊔, ▷}. (3)

✓ Solution

(a) §1. For n ≥ 0 add δ(q0, ⊔) = (q4, ⊔, R).

(b) Principle: change the leftmost a to X, then the first b to Y, then the first c to Z, return to X, repeat. When no a is left, verify only Ys and Zs remain up to the blank. 'Rounds' pair one a, one b and one c.

```
TM for {a^nb^nc^n | n ≥ 1}:
δ(q0, a) = (q1, X, R)
δ(q0, Y) = (q4, Y, R)
δ(q1, a,Y) = (q1, same, R)
δ(q1, b) = (q2, Y, R)
δ(q2, b,Z) = (q2, same, R)
δ(q2, c) = (q3, Z, L)
δ(q3, a,b,Y,Z) = (q3, same, L)
δ(q3, X) = (q0, X, R)
δ(q4, Y) = (q4, Y, R)
δ(q4, Z) = (q5, Z, R)
δ(q5, Z) = (q5, Z, R)
δ(q5, ⊔) = (q6, ⊔, R)
start q0; accepting/halting ['q6']; ⊔ = blank; 'same' = write the symbol read; q3 = return left, q4–q5 =
final check
```

```
IDs on aabbcc:
[q0]aabbcc ⊢ X[q1]abbcc ⊢ Xa[q1]bbcc ⊢ XaY[q2]bcc ⊢ XaYb[q2]cc ⊢ XaY[q3]bZc ⊢ Xa[q3]YbZc ⊢ X[q3]aYbZc ⊢
[q3]
XaYbZc ⊢ X[q0]aYbZc ⊢ XX[q1]YbZc ⊢ XXY[q1]bZc ⊢ XXYY[q2]Zc ⊢ XXYYZ[q2]c ⊢ XXYY[q3]ZZ ⊢ XXY[q3]YZZ ⊢
XX[q3]YY
ZZ ⊢ X[q3]XYYZZ ⊢ XX[q0]YYZZ ⊢ XXY[q4]YZZ ⊢ XXYY[q4]ZZ ⊢ XXYYZ[q5]Z ⊢ XXYYZZ[q5]⊔ ⊢ XXYYZZ⊔[q6]⊔
```

(c) Same idea with the third block of 0s marked Z; N = 0 is accepted by δ(q0, ⊔) = (q6, ⊔, R).

```
TM for {0^N1^N0^N | N ≥ 0}:
δ(q0, ⊔) = (q6, ⊔, R)
δ(q0, 0) = (q1, X, R)
δ(q0, Y) = (q4, Y, R)
δ(q1, 0,Y) = (q1, same, R)
δ(q1, 1) = (q2, Y, R)
δ(q2, 1,Z) = (q2, same, R)
δ(q2, 0) = (q3, Z, L)
δ(q3, 0,1,Y,Z) = (q3, same, L)
δ(q3, X) = (q0, X, R)
δ(q4, Y) = (q4, Y, R)
δ(q4, Z) = (q5, Z, R)
δ(q5, Z) = (q5, Z, R)
δ(q5, ⊔) = (q6, ⊔, R)
start q0; accepting/halting ['q6']; ⊔ = blank; 'same' = write the symbol read
```

(d) Round: mark an a (A), the first unmarked c (C), the first unmarked b (B), the first unmarked d (D); return to the last A and repeat. When no a is left the next symbol must be C and the rest must be C∗B∗D∗ then blank.

```
TM for {a^nc^nb^nd^n | n ≥ 0}:
δ(q0, ⊔) = (q8, ⊔, R)
δ(q0, a) = (q1, A, R)
δ(q0, C) = (q5, C, R)
δ(q1, a,C) = (q1, same, R)
δ(q1, c) = (q2, C, R)
δ(q2, c,B) = (q2, same, R)
δ(q2, b) = (q3, B, R)
δ(q3, b,D) = (q3, same, R)
δ(q3, d) = (q4, D, L)
δ(q4, D,b,B,c,C,a) = (q4, same, L)
δ(q4, A) = (q0, A, R)
δ(q5, C) = (q5, C, R)
δ(q5, B) = (q6, B, R)
δ(q6, B) = (q6, B, R)
δ(q6, D) = (q7, D, R)
δ(q7, D) = (q7, D, R)
δ(q7, ⊔) = (q8, ⊔, R)
start q0; accepting/halting ['q8']; ⊔ = blank; 'same' = write the symbol read
```

(e) Pair off: mark a 0 (X) and match it with an unmarked 1 (Y), or the other way round; repeat until a sweep finds no unmarked symbol. Accept when q0 reaches the blank having seen only X, Y.

```
TM for equal numbers of 0s and 1s:
δ(q0, X,Y) = (q0, same, R)
δ(q0, 0) = (q1, X, R)
δ(q0, 1) = (q2, X, R)
δ(q0, ⊔) = (q4, ⊔, R)
δ(q1, 0,Y) = (q1, same, R)
δ(q1, 1) = (q3, Y, L)
δ(q2, 1,Y) = (q2, same, R)
δ(q2, 0) = (q3, Y, L)
δ(q3, 0,1,X,Y) = (q3, same, L)
δ(q3, ⊔) = (q0, ⊔, R)
start q0; accepting/halting ['q4']; ⊔ = blank; 'same' = write the symbol read; q1 looks for a 1, q2 for a
0, q3 returns to the left end
```

(f) Compare the pieces symbol by symbol: mark the first unmarked symbol of w (X for a, Y for b), remember it in the state, skip to the first unmarked symbol after '−', check it is the same and mark it, return and repeat; finally check that no unmarked symbol is left after '−'.

```
TM for {w−w}:
δ(q0, X,Y) = (q0, same, R)
δ(q0, a) = (sa, X, R)
δ(q0, b) = (sb, Y, R)
δ(q0, -) = (chk, -, R)
δ(sa, a,b) = (sa, same, R)
δ(sa, -) = (ta, -, R)
δ(sb, a,b) = (sb, same, R)
δ(sb, -) = (tb, -, R)
δ(ta, X,Y) = (ta, same, R)
δ(ta, a) = (back, X, L)
δ(tb, X,Y) = (tb, same, R)
δ(tb, b) = (back, Y, L)
δ(back, a,b,X,Y,-) = (back, same, L)
δ(back, ⊔) = (q0, ⊔, R)
δ(chk, X,Y) = (chk, same, R)
δ(chk, ⊔) = (qa, ⊔, R)
start q0; accepting/halting ['qa']; ⊔ = blank; 'same' = write the symbol read; sa/sb remember a/b; ta/tb
check the right copy
```

For {w−w−w}: the same round marks one symbol in EACH of the three blocks (the remembered symbol must match in blocks 2 and 3) — states sa/sb → ta/tb → ua/ub → va/vb → back; when q0 reaches the first '−' check that blocks 2 and 3 are fully marked.

```
TM for {w−w−w}:
δ(q0, X,Y) = (q0, same, R)
δ(q0, a) = (sa, X, R)
δ(q0, b) = (sb, Y, R)
δ(q0, -) = (chk1, -, R)
δ(sa, a,b) = (sa, same, R)
δ(sa, -) = (ta, -, R)
δ(sb, a,b) = (sb, same, R)
δ(sb, -) = (tb, -, R)
δ(ta, X,Y) = (ta, same, R)
δ(ta, a) = (ua, X, R)
δ(tb, X,Y) = (tb, same, R)
δ(tb, b) = (ub, Y, R)
δ(ua, a,b) = (ua, same, R)
δ(ua, -) = (va, -, R)
δ(ub, a,b) = (ub, same, R)
δ(ub, -) = (vb, -, R)
δ(va, X,Y) = (va, same, R)
δ(va, a) = (back, X, L)
δ(vb, X,Y) = (vb, same, R)
δ(vb, b) = (back, Y, L)
δ(back, a,b,X,Y,-) = (back, same, L)
δ(back, ⊔) = (q0, ⊔, R)
δ(chk1, X,Y) = (chk1, same, R)
δ(chk1, -) = (chk2, -, R)
δ(chk2, X,Y) = (chk2, same, R)
δ(chk2, ⊔) = (qa, ⊔, R)
start q0; accepting/halting ['qa']; ⊔ = blank; 'same' = write the symbol read
```

(g) Principle: sweep right crossing out every second a; the number of a's must be even in every sweep (or exactly one a remains) — powers of two are exactly the numbers that stay even until they reach 1.

Our machine (states s, t, r) keeps the 1st, 3rd, 5th … a and crosses out the 2nd, 4th, …; if a sweep ends at b with an odd number > 1 of a's it halts without accepting; if a single a remains it accepts.

```
TM for {a^(2^n) b | n ≥ 0}:
δ(s0, X) = (s0, X, R)
δ(s0, a) = (s1, a, R)
δ(s1, X) = (s1, X, R)
δ(s1, a) = (t0, X, R)
δ(s1, b) = (qa, b, R)
δ(t0, X) = (t0, X, R)
δ(t0, a) = (t1, a, R)
δ(t0, b) = (r, b, L)
δ(t1, X) = (t1, X, R)
δ(t1, a) = (t0, X, R)
δ(r, a,X) = (r, same, L)
δ(r, ⊔) = (s0, ⊔, R)
start s0; accepting/halting ['qa']; ⊔ = blank; 'same' = write the symbol read
```

```
IDs on aaaab:
[s0]aaaab ⊢ a[s1]aaab ⊢ aX[t0]aab ⊢ aXa[t1]ab ⊢ aXaX[t0]b ⊢ aXa[r]Xb ⊢ aX[r]aXb ⊢ a[r]XaXb ⊢ [r]aXaXb ⊢
[r]⊔
aXaXb ⊢ ⊔[s0]aXaXb ⊢ ⊔a[s1]XaXb ⊢ ⊔aX[s1]aXb ⊢ ⊔aXX[t0]Xb ⊢ ⊔aXXX[t0]b ⊢ ⊔aXX[r]Xb ⊢ ⊔aX[r]XXb ⊢
⊔a[r]XXXb ⊢
 ⊔[r]aXXXb ⊢ [r]⊔aXXXb ⊢ ⊔[s0]aXXXb ⊢ ⊔a[s1]XXXb ⊢ ⊔aX[s1]XXb ⊢ ⊔aXX[s1]Xb ⊢ ⊔aXXX[s1]b ⊢ ⊔aXXXb[qa]⊔
```

(h) q0 on b stays (moves right), on a goes to q1; q1 on a accepts (two a's in a row) and on b returns to q0.

```
TM that scans right until it finds two consecutive a's:
δ(q0, b) = (q0, b, R)
δ(q0, a) = (q1, a, R)
δ(q1, b) = (q0, b, R)
δ(q1, a) = (qa, a, R)
start q0; accepting/halting ['qa']; ⊔ = blank; 'same' = write the symbol read; halts in qa on reading the
second a; if no aa the head runs off the input and halts at ⊔ with no move
```

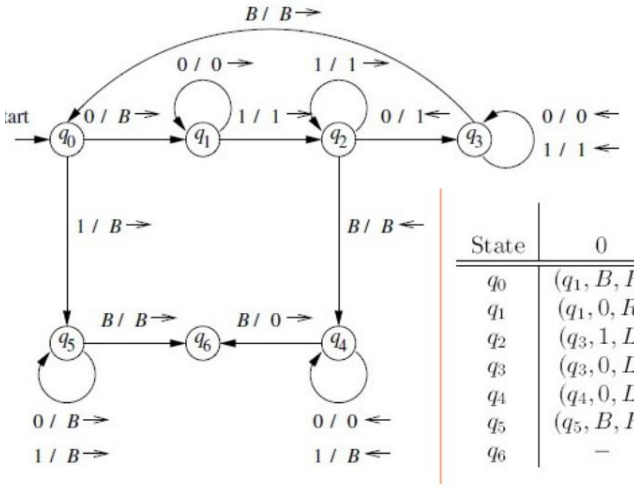
Lesson 13 — TMs Computing Functions

Slides 71–115 + TF problems

Turing's original view: integer-valued functions with integers in unary (blocks of one symbol). The machine changes the lengths of blocks or builds new ones. The final tape contents (with infinite blanks around) is the output.

1. Monus m ∸ n = max(m − n, 0) (sl. 72–115)

Input 0^m 1 0^n; output 0^(m∸n). M repeatedly finds the leftmost 0, replaces it by a blank, searches right for the 1 and on past it to a 0 which it replaces by 1; then returns left (sl. 75). Two exits: (1) searching right for a 0 it meets a blank — the n zeros are used up and n + 1 of the m zeros have been blanked; M turns the n + 1 ones into one 0 and n blanks (m ≥ n); (2) the first m zeros are all gone — the result is 0 and the rest is blanked (sl. 76).



State	Symbol 0	1	B
q_0	(q_1, B, R)	(q_5, B, R)	–
q_1	$(q_1, 0, R)$	$(q_2, 1, R)$	–
q_2	$(q_3, 1, L)$	$(q_2, 1, R)$	(q_4, B, L)
q_3	$(q_3, 0, L)$	$(q_3, 1, L)$	(q_0, B, R)
q_4	$(q_4, 0, L)$	(q_4, B, L)	$(q_6, 0, R)$
q_5	(q_5, B, R)	(q_5, B, R)	(q_6, B, R)
q_6	–	–	–

Figure 13.1 — Transition diagram and table of the monus TM (sl. 105–106)

```
Monus TM (same states as the slides):
δ(q0, 0) = (q1, ⊔, R)
δ(q0, 1) = (q5, ⊔, R)
δ(q1, 0) = (q1, 0, R)
δ(q1, 1) = (q2, 1, R)
δ(q2, 1) = (q2, 1, R)
δ(q2, 0) = (q3, 1, L)
δ(q2, ⊔) = (q4, ⊔, L)
δ(q3, 0,1) = (q3, same, L)
δ(q3, ⊔) = (q0, ⊔, R)
δ(q4, 1) = (q4, ⊔, L)
δ(q4, 0) = (q4b, 0, L)
δ(q4, ⊔) = (q6, 0, R)
δ(q4b, 0) = (q4b, 0, L)
δ(q4b, ⊔) = (q6, 0, R)
δ(q5, 0,1) = (q5, ⊔, R)
δ(q5, ⊔) = (q6, ⊔, R)
start q0; accepting/halting ['q6']; ⊔ = blank; 'same' = write the symbol read; q0 start of a round, q1 find
1, q2 find 0, q3 return, q4 clean up, q5 erase all, q6 halt
```

```
IDs on 0000100:
[q0]0000100 ⊢ ⊔[q1]000100 ⊢ ⊔0[q1]00100 ⊢ ⊔00[q1]0100 ⊢ ⊔000[q1]100 ⊢ ⊔0001[q2]00 ⊢ ⊔000[q3]110 ⊢
⊔00[q3]011
0 ⊢ ⊔0[q3]00110 ⊢ ⊔[q3]000110 ⊢ [q3]⊔000110 ⊢ ⊔[q0]000110 ⊢ ⊔⊔[q1]00110 ⊢ ⊔⊔0[q1]0110 ⊢ ⊔⊔00[q1]110 ⊢
⊔⊔001[
q2]10 ⊢ ⊔⊔0011[q2]0 ⊢ ⊔⊔001[q3]11 ⊢ ⊔⊔00[q3]111 ⊢ ⊔⊔0[q3]0111 ⊢ ⊔⊔[q3]00111 ⊢ ⊔[q3]⊔00111 ⊢
⊔⊔[q0]00111 ⊢ ⊔⊔
⊔[q1]0111 ⊢ ⊔⊔⊔0[q1]111 ⊢ ⊔⊔⊔01[q2]11 ⊢ ⊔⊔⊔011[q2]1 ⊢ ⊔⊔⊔0111[q2]⊔ ⊢ ⊔⊔⊔011[q4]1⊔ ⊢ ⊔⊔⊔01[q4]1⊔⊔ ⊢
⊔⊔⊔0[q4]1
⊔⊔⊔ ⊢ ⊔⊔⊔[q4]0⊔⊔⊔⊔ ⊢ ⊔⊔[q4b]⊔0⊔⊔⊔⊔ ⊢ ⊔⊔0[q6]0⊔⊔⊔⊔
```

Exam Approach — Function TMs

1. Fix the input/output format (unary 0^m 1 0^n, 1^a 0 1^b, binary #N, …).
2. Describe one round and the invariant; list the exits (finished, one input exhausted).
3. Give the table; check with the smallest cases (0, 1 or equal arguments).
4. Check the final tape is clean (only the answer, rest blank).

Practice — Lesson 13

(a) Actual — 2015-16 Q8(a). (b) Actual — 2011-12 Q1(a). (c) Actual — 2016-17 Q5(b). (d) Actual — 2011-12 Q3(a) / 2012-13 Q3(c). (e) Constructed (slides' 4 ∸ 2 and 2 ∸ 4 traces).

(a) Design a TM that computes a ∸ b = max(b − a, 0). Initially the tape contains a followed by b, both written as consecutive 1s separated by a 0; at the end only the answer, as consecutive 1s, remains. Describe the working principle. (13)

(b) Design a TM computing x − y (x ≥ y) with N represented in unary as 1^(N+1). Assume the tape holds 1^(x+1) 0 1^(y+1) and the answer is 1^(x−y+1). (8⅓)

(c) TM that adds 1 to a binary number N. The tape holds # followed by N (e.g. #101) with the head on #; halt after converting N to N + 1; you may replace # by 0 or 1. Explain the principle. (20)

(d) TM that, starting on #w (w ∈ {0,1}∗), writes #0^n if |w| = n is even and #1^n if n is odd. (8⅓)

(e) Run the monus TM on 0^4 1 0^2 and on 0^2 1 0^4.

✓ Solution

(a) Principle: pair off ones. Mark (X) the leftmost unmarked 1 of a, go right over the 0, mark the first unmarked 1 of b, return and repeat. If a's ones run out first (the machine reads 0 in q0) erase all Xs and the 0, keeping the remaining ones: they number b − a. If b runs out first (q2 reads blank) then b ≤ a: erase everything — the answer is the empty tape (0). The result 1^(b−a) is contiguous.

```
TM for max(b − a, 0):
δ(q0, X) = (q0, X, R)
δ(q0, 1) = (q1, X, R)
δ(q0, 0) = (q9, 0, L)
δ(q1, 1) = (q1, 1, R)
δ(q1, 0) = (q2, 0, R)
δ(q2, X) = (q2, X, R)
δ(q2, 1) = (q3, X, L)
δ(q2, ⊔) = (q7, ⊔, L)
δ(q3, X,0,1) = (q3, same, L)
δ(q3, ⊔) = (q0, ⊔, R)
δ(q9, X) = (q9, ⊔, L)
δ(q9, ⊔) = (q10, ⊔, R)
δ(q10, ⊔) = (q10, ⊔, R)
δ(q10, 0) = (q11, ⊔, R)
δ(q11, X) = (q11, ⊔, R)
δ(q11, 1) = (q6, 1, R)
δ(q11, ⊔) = (qh, ⊔, R)
δ(q6, 1) = (q6, 1, R)
δ(q6, ⊔) = (qh, ⊔, R)
δ(q7, X,0,1) = (q7, ⊔, L)
δ(q7, ⊔) = (qh, ⊔, R)
start q0; accepting/halting ['qh']; ⊔ = blank; 'same' = write the symbol read; q1 walk to the 0, q2 find an
unmarked b-one, q3 return, q9–q11 clean up when a is exhausted, q7 erase all
```

(b) Same pairing: each round marks one 1 of the x-block and one 1 of the y-block. The y-block has y + 1 ones; after them the machine discovers the y-block is exhausted. If the x-block still has an unmarked 1 it was marked wrongly in the last round (the over-marked X); the clean-up erases the markers and turns the last two (or one, when x = y) marked cells of the left block into 1s so that exactly x − y + 1 ones remain.

```
TM for 1^(x+1) 0 1^(y+1) → 1^(x−y+1):
δ(q0, X) = (q0, X, R)
δ(q0, 1) = (q1, X, R)
δ(q0, 0) = (q8, 0, R)
δ(q1, 1) = (q1, 1, R)
δ(q1, 0) = (q2, 0, R)
δ(q2, X) = (q2, X, R)
δ(q2, 1) = (q3, X, L)
δ(q2, ⊔) = (q4, ⊔, L)
δ(q3, X,0,1) = (q3, same, L)
δ(q3, ⊔) = (q0, ⊔, R)
δ(q8, X) = (q8, X, R)
δ(q8, ⊔) = (q9, ⊔, L)
δ(q9, X) = (q9, ⊔, L)
δ(q9, 0) = (q9b, ⊔, L)
δ(q9b, X) = (q6, 1, L)
δ(q4, X) = (q4, ⊔, L)
δ(q4, 0) = (q4b, ⊔, L)
δ(q4b, 1) = (q5, 1, L)
δ(q4b, X) = (q6a, 1, L)
δ(q5, 1) = (q5, 1, L)
δ(q5, X) = (q6a, 1, L)
δ(q6a, X) = (q6, 1, L)
δ(q6, X) = (q6, ⊔, L)
δ(q6, ⊔) = (qh, ⊔, R)
start q0; accepting/halting ['qh']; ⊔ = blank; 'same' = write the symbol read
```

(c) Principle (binary increment): go to the right end; flip trailing 1s to 0 moving left; the first 0 becomes 1 and the machine halts; if the # is reached (all 1s), replace # by 1.

```
TM adding 1 to a binary number written after #:
δ(q0, #) = (q1, #, R)
δ(q1, 0,1) = (q1, same, R)
δ(q1, ⊔) = (q2, ⊔, L)
δ(q2, 1) = (q2, 0, L)
δ(q2, 0) = (qh, 1, R)
δ(q2, #) = (qh, 1, R)
start q0; accepting/halting ['qh']; ⊔ = blank; 'same' = write the symbol read
```

#101 → q0 on #, q1 skips 101, blank → q2 on 1 → 0 (left), 0 → 1: #110 ✓; #111 → #000 and then # → 1: 1000 ✓ (checked on all N up to length 6).

(d) Pass 1 (left to right): flip a parity state between 'even' and 'odd' while skipping symbols. At the blank, move left and overwrite every symbol with 0 (even) or 1 (odd) until # is reached; halt. The parity is stored in the state (storage in the finite control).

```
TM for #w → #0^n (n even) / #1^n (n odd):
δ(q0, #) = (pe, #, R)
δ(pe, 0,1) = (po, same, R)
δ(po, 0,1) = (pe, same, R)
δ(pe, ⊔) = (we, ⊔, L)
δ(po, ⊔) = (wo, ⊔, L)
δ(we, 0,1) = (we, 0, L)
δ(we, #) = (qh, #, R)
δ(wo, 0,1) = (wo, 1, L)
δ(wo, #) = (qh, #, R)
start q0; accepting/halting ['qh']; ⊔ = blank; 'same' = write the symbol read
```

(e) 4 ∸ 2 = 2: tape ends 00; 2 ∸ 4 = 0: tape ends blank (exit 2: all remaining 1s and 0s are blanked). Both runs were checked by simulation (all m, n up to 7).

Quick Notes — TM Basics

Revision points for Lessons 11–13.

L11 · Definition

- M = (Q, Σ, Γ, δ, q0, B, F); δ(q, X) = (p, Y, D), D ∈ {L, R}; head must move; B ∈ Γ − Σ.
- ID: X1…Xi−1 q Xi…Xn; boundary cases: left of X1, blank written at an end disappears.
- L(M): reach an accepting state; halt = no move; RE = TM-accepted languages. Γ ≠ Σ; blank may be written.

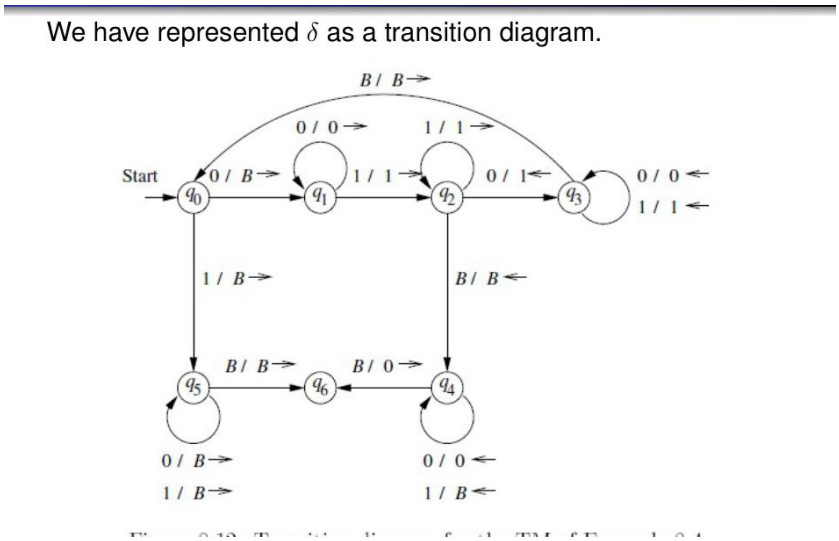
L12 · Deciders

- Marking: X/Y/Z for matched symbols; one round per group; accept when only marks remain.
- 0^n1^n: 5 states q0–q4 (q0: 0→X; q1 over 0,Y; 1→Y; q2 back; q3 check Y∗␣).
- a^nb^nc^n: X, Y, Z; w−w: remember a/b in the state; equal counts: X + opposite Y; 2^n: cross out every 2nd a.

L13 · Functions

- Monus 0^m10^n: erase leftmost 0, find 1, turn first 0 after it into 1, return; exits: blank (m ≥ n) or no 0 left (result 0).
- Increment: right end, 1→0 leftwards, first 0 (or #) → 1. Parity: state flips, rewrite symbols with the parity digit.

Key Diagrams to Reproduce



0^n1^n TM diagram (sl. 105)